

WHERE HOLLYWOOD MEETS BIOLOGY



A new breed of multimedia scientists are emerging.
<http://goo.gl/mwMwg>

LEAVE ME THE \$%&K ALONE!

Just kidding. What you should know about though is the brand new social browser called Rockmelt that's creating waves in Silicon Valley (California). It's invitation based only right now, but looks very promising in the social space.
<http://goo.gl/uWN46>

WORKSPACES OF THE RICH AND FAMOUS

From the creator of Dilbert to Steve Jobs, take a sneak peek into the private workspaces of these famous people. Who knows, maybe you could pick up a tip or two and have your own 'famous workspace' featured in this space in the future.
<http://goo.gl/mStQ3>

EXPORT YOUR FACEBOOK FRIENDS



Facebook not as appealing anymore? Breaches in privacy not your cup of tea? Want to break free without losing all your friends?
<http://goo.gl/fw8Ug>

How Many Cores Is Too Many?

- By Matthew Murray



Copyright © 2010
Ziff Davis Publishing
Holdings Inc.

Ever since the first dual- and quad-core CPUs were released several years ago, the procedure for buying processors has involved counting the number of cores in the CPU, counting the number of dollars in your wallet, and then doing your best to match up the values. After all, chips that haven't been overclocked haven't gotten much faster than 3.3 GHz or 3.4 GHz, so despite a bit of variance here and there, you've generally been able to feel secure that the more cores your CPU contained, the faster it would perform overall.

This has been acceptable thinking up until now—after all, it's only this year, with the release of Intel's Core i7-980X and Core i7-970 and AMD's Phenom II X6 series (including the 1090T, the 1055T, and the 1075T), that

the number of cores on consumer CPUs has outgrown what you can count using just the fingers on one hand. But technology never stops advancing, and some recent research is showing that current advice to buy the most cores you can afford may not remain practical for much longer.

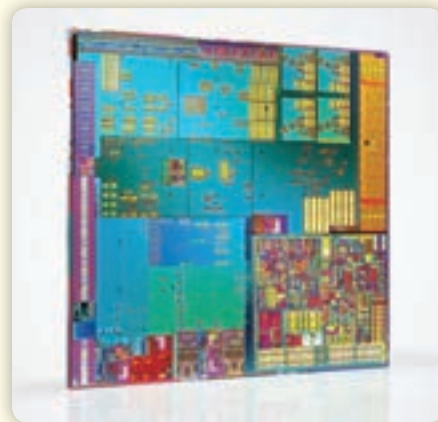
A group of MIT researchers are presenting a paper today at the USENIX Symposium on Operating Systems Design and Implementation in Toronto, titled "An Analysis of Linux Scalability to Many Cores," that details research about how very large numbers of cores affect processing performance. The paper may deal with Linux specifically, but it's a helpful reminder to everyone about the challenges facing hardware and software design in today's computing world—and how important it is that these problems get solved soon.

The problem, according to the researchers, starts appearing with systems bearing numbers of cores in the dozens. They built a system in which eight six-core chips effectively mimicked one 48-core chip, then observed what happened in a lengthy series of tests. The large number of cores may have made for a blazing-fast system, but it was still slower than it should have been. The reason? This story from MITnews explains:

In a multicore system, multiple cores often perform calculations that involve the same chunk of data. As long as the data is still required by some core, it shouldn't be deleted from memory. So when a core begins to work on the data, it ratchets up a counter stored at a central location, and when it finishes its task, it ratchets the counter down. The counter thus keeps a running tally of the total number

of cores using the data. When the tally gets to zero, the operating system knows that it can erase the data, freeing up memory for other procedures.

As the number of cores increases, however, tasks that depend on the same data get split up into smaller and smaller chunks. The MIT researchers found that the separate cores were spending so much time ratcheting the counter up and down that they weren't getting nearly enough work done.



According to the paper, the problems were often due to issues with cache:

Many scaling problems manifest themselves as delays caused by cache misses when a core uses data that other cores have written. This is the usual symptom both for lock contention and for contention on lock-free mutable data. The details depend on the hardware cache coherence protocol, but the following is typical. Each core has a data cache for its own use. When a core writes data that other cores have cached, the cache coherence protocol forces the write to wait while the protocol finds the cached copies

